Julius-Maximilians-Universität Würzburg · CTB Chair for Computational and Theoretical Biology · Games Engineering · IOF Imaging Core Facility · Biozentrum Universität Würzburg

How do you build a virtual *C. elegans*?

Towards an Open Interactive Real-Time Platform for Connectome-Driven Behavioral Simulation of *C. elegans*

Felix Fink · Simon Stöckl · Marie Holfelder · Sarah Wolpold · Thomas Dandekar · Christian Stigloher · Sebastian von Mammen · Philip Kollmannsberger

But why. ... me?

Easy. Utterly unremarkable to the uninitiated, yet at 1 mm of length you're a superstar of the life sciences - a literal super model. Every adult hermaphrodite is built from exactly 959 somatic cells, no exceptions, and each one can be traced back through its lineage to the zygote. the same body plan, cell for cell, in every individual. Which is why you've been poked at by geneticists, neuroscientists, ethologists, medics and computational biologists alike.

Ha! That's why the illustrator gave me this ridiculous brain? Because ist small, compact and accessible? Unremarkable, my intestine.....

Yes.

Awesome! Can my brain be mapped and simulated?

In fact your nervous system - at least the adult hermaphrodite - was the first one to be fully mapped. 302 neurons, 95 body wall muscles, around 7000 connections. Data that dates back to 1986 and ever since then, this data has been subject to computational studies alongside experimental ones. In the 2010s the OpenWorm project went further and developed a connectome simulator (c302) , a dataformat for exchanging neuronal data as well as a particle-based physics solver for a simulated body in a virtual viscous environment (Sibernetic).

That's cool! But why build a simulator again?

Tools exist, but what's lacking is an interdisciplinary playground. Sure, this project comes from biology, but it touches computer science, neuroscience, philosophy and even art. And right now, the entry barrier to OpenWorm for people unfamiliar with biology or programming etc. is quite high, while the actual technical entry barrier has been lowered. PCs are much more powerful now, rendering complex videogames in 4K in real time. Likewise, access to development tools, learning ressources and online communities has increased.

So what's the benefit of bringing different backgrounds together?

Different approaches, different ideas and questions from different vantage points. Even before a worm can be solved its precisely this exchange that is worth building towards to .To investigate the connectome not only data is necessary, but also tools for speculation and hypothesis. One might think a connectome alone might be a blueprint to a working brain, implement it an theres the virtual animal. But its just thumbnail, really. Just like you are not a worm, but a sketch, a crude representation of one.

Ceci n'est pas un ver ?

Nobody will build a virtual *C. elegans* alone.
But a basic toolbox is possible: a body, an environment, a nervous system on real data that senses both , even crudely. And how do you interact with a virtual nervous system? How do you make its topology and dynamics tangible? To adress these questions is why it seems worthwhile to use tools that are designed to develop immersive, real time applications and attract not only scientists and engineers, but artists and UX designers as well.

Well, thanks for the worm-up!

Body & Muscles

Real worms have bodies and so must virtual ones. The word itself describes a body - elongated, limbless, invertebrate. No body, no worm. To simulate a virtual worm it needs shape & volume , internal & external forces and a physics solver maintianing the first under the influence of the second.

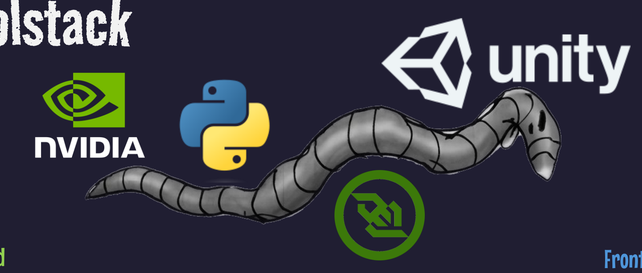
Shape
- Cylindrical base mesh with icosphere caps
- Mesh resolution, tapering and bulging are adjustable parameters to vary worm shape

Volume
- A tetrahedral mesh is generated from the initial surface mesh

Muscles
- Tangential edges are marked as contractile fibres. Several fibres are grouped as one muscle, 24 muscles per quadrant.

Solver
- XPBD. Initializes mesh edges as distance constraints and tetrahedra as volume constraints.
- Muscle forces are generated from the muscle system and are provided with external forces like gravity or drag

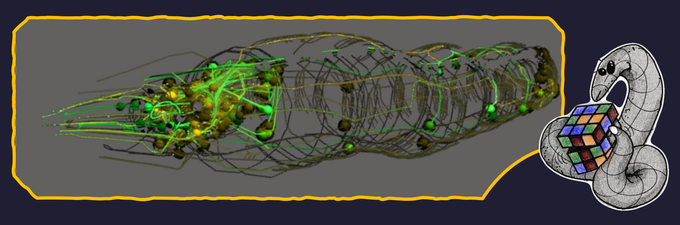
Toolstack



Backend
- Python
- NVIDIA Warp, Framework for GPU accelerated computational physics. Used to implement the necessary solvers (Softbody Physics, Fluid dynamics, neuron dynamics)
- Omits the necessity to implement CUDA or GPU code.
- Acccessible to data scientists, bioinformaticians and anyone with python experience.
- Simulation Data (positions, velocities meshes) are streamed to the Client through a websocket.

Frontend
- We use Unity3D as Client
- Tools for designing User interactions and UI
- Rendering Pipelines and Shaders
- Visualization and appeal
- Large Online communities and learning Ressources
- Changes to parameters, interaction events etc are sent to the simulator via websockets.

Brain and pattern generation



How to represent a brain in silico? There is:

- **Morphology** : soma positions, volume, dendrite shapes and path, axon paths
- **Connectivity / Topology.**: Connections between neurons. Chemical or electrical synapses. Number of synapses as proxy for strength
- **Electrophysiology.** Membrane potentials and dynamics, resting potentials, intracellular ion concentrations
- **Neurotransmitters.** The „sign" of the connections

Where does the data come from?
We use OpenWorms NeuroML files^, which store morphology and electrophysiology. Morphology is rendered in PyVista or Unity

How to simulate the system?
We use the Leaky-Integrate-And-Fire model to simulate action potential dynamics for 302 neurons

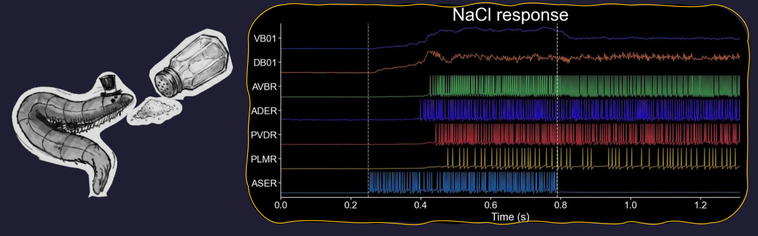
$$\tau_m \frac{dV}{dt} = -(V - V_{rest}) + R_m I(t)$$

How to generate stable patterns to test locomotion and physics?
We also use a pattern generator to produce a sinusoidal signal to drive the dorsal and ventral muscles antagonistically and introduce a variable travelling wave^.

Using the SPH-weigh kernels to simulate diffusion among local neighbors.
How does the sensing work? The chemosensitive neurons morphology is embedded and achored to the mesh. The dendrites act as probes, that sample either the signal field at their grid location or sample signals using the SPH weighting kernels

Chemosensation

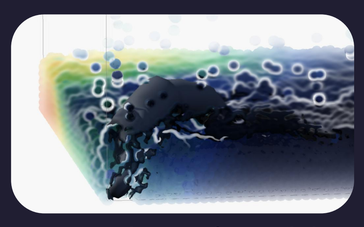
How are chemical signals represented?
- **Grid-based diffusion.** The simulation domain is discretized using a 3D grid. Chemicals diffuse on this grid, thus creating signal gradients.
- **Particle-based diffusion.** Instead of a fixed grid, diffusion is computed



When presented with a diffusive salt stimulus, the known salt-sensitive neuron ASER responds in our simulation and appears to elicit an activation cascade. Interneurons such as AVBR and ADER increase activity.

Motorneurons VB01 and DB01 are activated. The response of PVDR and PLMR suggest, that the body deformation induced by the motorneurons, is sensed and contributes to the PVDR, PLMR signal

Fluid and environment



What's the environment of a worm like?
C. Elegans lives under low-Reynolds conidtions in fluid, viscous environments like rotting fruit and fluid films

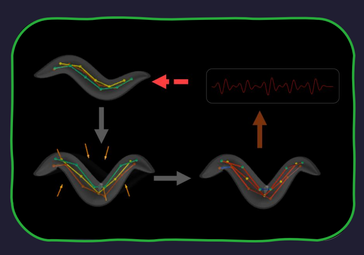
How does the environment affect its' propulsion? Depending on the viscosity C. elegans is either swimming or crawling, characterized by the beating frequency and bending wavelength.

How to approximate these environments?

Resistive force theory. No fluid at all. Each surface element splits its velocity into normal and tangential components; the ratio is the medium. Cheap, and enough for locomotion — but the worm has no effect on its surroundings

Smoothed Particle Hydrodynamics. The fluid is discretized into particles, each carrying mass, velocity, density and pressure. Any field quantity is recovered by summing over the neighbours within a radius, kernel-weighted by distance. This updates particle density and local pressure and viscosity.

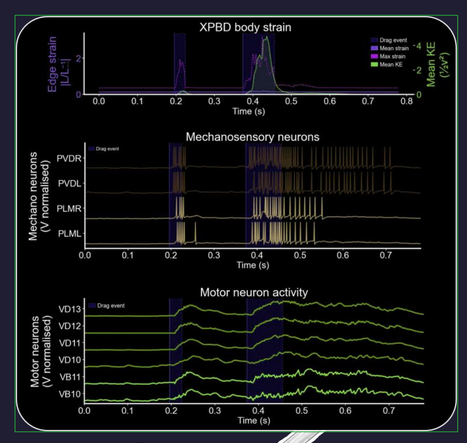
Mechanosensation

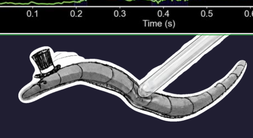


Touch sensation is an important sensor modality to C. Elegans and it can distinguish between *gentle* and *harsh* touch. Both are necessary in exploration and avoidance behaviour.

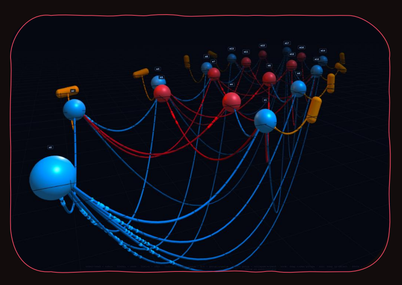
How is touch sensation implemented?
The morphology of the mechanosensitive neurons is crucial for this. We embed the morpholgy within the body and skin it to the surface mesh. Each segment has a reference rest length to which the current segment length is compared. This yields an error, which defines a small input current. Input currents are integrated over all segments and used in the neuron simulation

The panel below shows the mechanosensitive systems' response to user induced mechanical perturbation. Here, the user grabs the worms surface and pulls it around (purple areas). Mechanosensory neurons PVDR/L and PLML/R are visualized here and display spiking activity shortly after mechanical perturbation. Motoneurons of the ventral nerve chord increase activity shortly after.





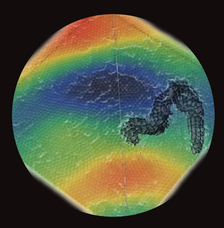
Interactive Connectomics



No one likes to stare at spreadsheets. So we're working on engaging ways to interact with the virtual worms' wiring.
- Neurons are represented as spheres color coded by type. (excitatory, inhibitory
- Capsules represent muscles and contract when activated by the neurons
- Connections can be drawn freely, adjusted, muted)
- Motifs can be grouped and copy-pasted
- A connectivity matrix of the current setup can be displayed

Visualization

Presentation matters. Here we used standard shader techniques for surfaces and fluids as well as camera effects.

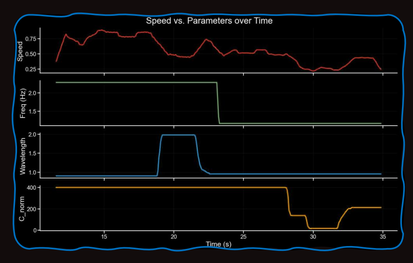


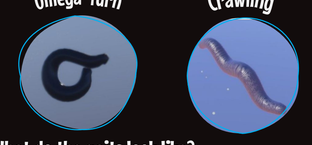
Screen-space fluid rendering. Used to render particle clouds as a smooth surface. Particles are rendered as spheres, blurred by depth. Normals are recovered from depth. Light absoprtion, scattering and reflection are added, to make the fluid appear realistic.

Coloring schemes can be adjusted to visualize any quantity: Velocity, density, pressure, etc
Camera effects. We used a Depth-of-Field effect, chromatic abberation, lens distorion and a slight vignetting effect to create a macro lens effect. This should mimic, what an observer would see, looking through a binocular microscope.
Rendering the worm. For the worm procedural textures and shaders were used, which simulate physical parameters like the roughness, reflectance, transmissivity or normals for the interaction with light.

Looks like a worm, crawls like a worm?

What is measured ?
Locomotion speed while beating wavelength, CPG frequency and normal drag (C_norm) are varied at runtime. Increasing the wavelength from 0.7 to 2.0 slows the worm. Lowering the frequency slows it slightly. Making the drag isotropic slows it most — undulatory propulsion needs the anisotropy





What do the gaits look like?
Both gaits are generated from adjusting the parameters of the pattern generator .

Omega-Turn: Created by setting a high wavelength and low frequency. This leads to the entire dorsal and ventral side contracting slowly, creating coiling
Crawling: Low wavelength and higher frequency create the characteristic crawling shape. Under RFT conditions the worm moves directionally.